The Journal of Open Source Software

LabRPS: Laboratory for Random Phenomena Simulation

Koffi Togbenou [1,2¶], **Gabin Koto N'Gobi** [2], **Chuanjin Yu** [3], **Dossa Didier Boko-haya** [1,2], **Eklou Eric Hiheglo** [4], **Agbazo Médard Noukpo**[6], **Guy Hervé Houngue** [2], **Hagninou Elagnon Venance Donnou** [2], **and Patrick Arnaud Wandji Zoumb** [5]

1 China Railway Major Bridge Reconnaissance & Design Institute Co., Ltd., Wuhan 430056, China **2** Laboratoire de Sciences des Matériaux et Modélisation (LaSMMo), Faculté des Sciences et Techniques (FAST), Université d'Abomey-Calavi, Cotonou, Bénin. **3** Department of Bridge Engineering, Southwest Jiaotong University, 610031, Chengdu, Sichuan, China **4** Department of Mechanical Engineering, Systems for Hybrid-Additive Process Engineering Lab, Pennsylvania State University, USA **5** School of Civil Engineering, National Advanced School of Public Works, Yaoundé, Cameroon **6** Département d'Hydrologie, Université de N'zérékoré, Guinée ¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Richard Littauer

Submitted: 18 February 2025
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

LabRPS is an open-source computational tool designed for the numerical simulation of random phenomena. It leverages a modular framework enabling reusable of algorithm components, allowing for efficient simulation of various stochastic processes, including wind velocity, seismic ground motion, sea surface behaviour, road surface roughness, temperature fluctuations, and random material properties, among others.

LabRPS is primarily intended to assist researchers in this field to quickly implement new simulation methods programmatically in their research work based on existing works by providing a flexible platform for implementing and testing new simulation's methods and also for accessing existing implementations that have already been stored. It enables engineers and scientists to generate more realistic simulations of random phenomena, enhancing the accuracy, the computation time consumption and the memory usage of their simulation models. Additionally, it serves as a learning tool for students and newcomers to the field, offering a streamlined approach to grasping complex concepts.

The tool is highly modular and seamlessly integrates with an advanced data visualizer, enabling the creation of high-quality 2D and 3D visualizations for simulation results, as well as data imported from files, manually entered, or derived from formulas. These data visualizations are optimized for academic publications, empowering users to effectively communicate their findings with precision and clarity. Figure 1 displays the graphical user interface of LabRPS.

The Journal of Open Source Software

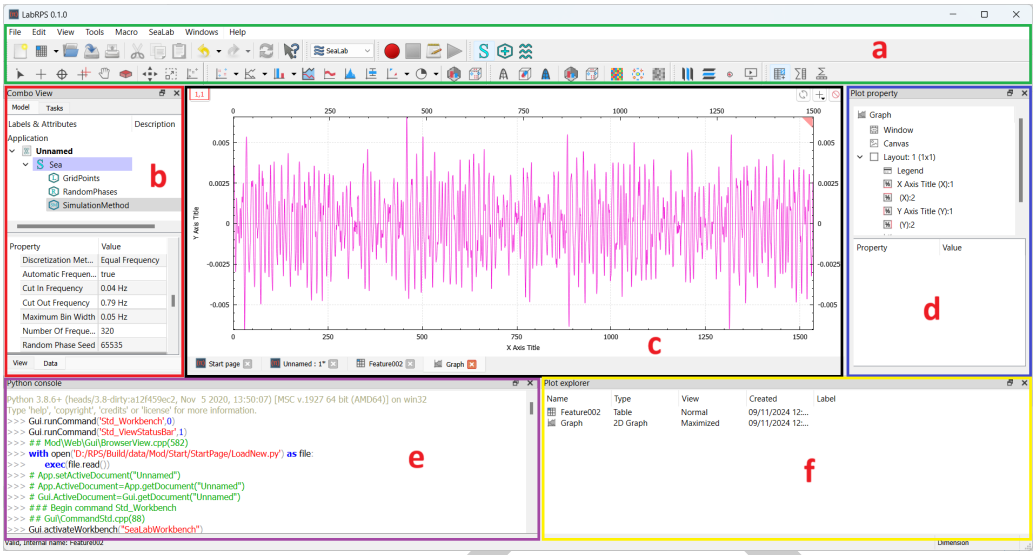

Figure 1: LabRPS Interface: (a) the toolbar and menu area that allows users to invoke various commands, (b) the combo view, comprising the tree view, task panel, and property editor, (c) the main view area, featuring multiple tabbed windows, (d) the plot editor view, enabling users to edit plots, (e) the Python console, running an instance of the Python interpreter for creating and modifying objects and their properties, (f) the plot explorer, which lists all created plots.

Statement of Need

Most of the problems that engineers, and scientists face involve random phenomena or randomness which is the lack of predictability. In many situations, it is desired to numerically represent those random phenomena. For example: (a) numerical correlated stochastic wind velocity is required for the buffeting analysis of flexible long span cable supported bridges, (b) boundary layer turbulence is required for the analysis of panel vibrations of aircraft and submarines, (c) ocean waves need to be numerically represented for the analysis of ship oscillations, (d) randomness in material must be well represented for the analysis of the effect of randomness in geometrical configuration of structural component due to fabrication errors on the vibration and buckling eigenvalues, (e) numerical representation of random surface roughness of airport run-way is required for the analysis of aircraft vibration caused by the roughness.

Currently, there are several methods available to numerically represent those random phenomena. Most of these methods are based on the Monte Carlo simulation (Kareem, 2008; Spanos & Zeldin, 1998) which can be achieved by frequency domain approaches such as linear filtering (Deodatis & Shinozuka, 1988; Yousun Li & Kareem, 1990; MIGNOLET & SPANOS, 1989) approaches or spectral representation method (Chen & Letchford, 2004; Grigoriu, 1993; Yongle Li et al., 2017; Yousun Li & Kareem, 1991; Shinozuka & Jan, 1972; Togbenou et al., 2016, 2018). From the previous studies, the complexity associated to these methods are undoubtable. Application results in a time-consuming task due to the significantly large number of simulated processes. This can even lead to a higher demand in terms of computer memory. In the recent years, effort has been made to provide these issues with suitable solutions. However, the solutions involve complex physics and mathematical tools which make their implementations in a computation program very difficult and time consuming.

It has been noticed that computer programs that offer suitable features for the computation works related to the numerical simulation of random phenomena are very few. In most cases, the simulation approach is implemented in the commercial software such as MATLAB which is a very powerful tool that offers number of solutions in numerical computation. The numerical

DRAFT

simulation of random phenomena using Matlab requires some programming skills. It may be time consuming to be involved with programming tasks each time a simulation is required. We observed that researchers, who often build upon the works of other researchers during the study of new simulation approaches, frequently invest significant time and effort merely to replicate the existing computational results. Reproducing computational research presents challenges in this field. A well-designed computer program that makes scientific computations reproducible, storable and accessible will be of a big benefit. A program that helps researchers and engineers to implement quickly new simulation approaches without having to implement or to understand the programming background of the existing simulation approaches is needed. To address these problems, we have developed LabRPS, a plugin-based system designed to facilitate the implementation, storage, and access of simulation algorithm (source codes and binaries) in order to accelerate the development of new simulation algorithms in this field. Additionally, LabRPS features an intuitive user interface that enables users to numerically simulate random phenomena without the need for coding.

Programming Language and Resources

Processing speed is especially important when dealing with numerical simulation of random phenomena which usually involves complex equations and algorithms with many iterations. Matlab is a common tool for numerical simulation of random phenomenon. It combines its ability to compute complex formulas with its graphical capabilities, which allows users to visualize the simulation data. Though Matlab is a powerful tool in visualization and computation of random phenomenon simulation, it does restrict developers in the ability to implement for example a serious plugin system as desired in the computer tool proposed in this study. Other programming languages, such as C++ and Python, offer a more versatile environment for developers to implement complex features such as a plugin architecture.

Python programming language is currently the most popular language worldwide and can be a good solution for the implementation of a plugin system of any complexity. However, Python is known for its performance limitation disadvantages where it (as Interpreted Language) can lead to slower computation speed compared to compiled languages like C or C++. This may be a concern when handling large datasets or complex computations which are the situations usually encountered in the field of random phenomenon numerical simulation. Therefore, in this study, we propose a combination of both languages which will give the advantage to ally the speed of C++ to the flexibility of Python. According to the current practice, Matlab is extensively used for these purposes. Therefore, LabRPS is designed in a way to allow access to its simulation workflow from Matlab. Any time-consuming tasks are coded in C++ to allow high computation speed performance. Python and Matlab are used as scripting languages to allow automation and most importantly to initiate time-consuming C++ based simulation algorithm.

One of the primary objectives of this research is to promote the development of new simulation algorithms by building upon existing works. In alignment with this philosophy, a significant component of LabRPS's codebase is directly derived from or adapted from FreeCAD (Yorik & others, 2022), an open-source parametric 3D CAD modeler renowned for its modular and extensible architecture. FreeCAD's core features have been instrumental in shaping the functionality of LabRPS.

Workbench Framework and Scripting Interface

In LabRPS, a Workbench is a set of tools, features, and functions that are organized based on specific tasks or workflows. Each workbench in LabRPS is tailored to a particular aspect of the simulation process, providing an optimized interface for users working on a particular type of random phenomenon. The workbench concept allows users to switch between different

tools depending on the stage or needs of the simulation. Examples of LabRPS Workbenches:

- **WindLab Workbench:** Focuses the simulation of random wind velocity
- **SeismicLab Workbench:** For the simulation of seismic ground motion
- **SeaLab Workbench:** Enables users to simulate random sea surface elevation

By switching between workbenches, users can tackle different aspects of a simulation more effectively, providing a tailored and efficient workflow for diverse simulation tasks, etc.

LabRPS scripting (see Figure 2) using Python is a powerful way to automate and customize your simulation process, especially for parametric features and repetitive tasks. Whether you're a hobbyist or a professional, understanding scripting opens up endless possibilities for enhancing your LabRPS experience. It should be noted that, this feature has been adapted from FreeCAD project.

```python
installResuslt = WindLab.installPlugin("WindLabPlugin")

# Document Creation
doc = LabRPS.newDocument()

# Simulation creation
sim = WindLabObjects.makeSimulation(doc, "Sim")

# Simulation points
loc = WindLabObjects.makeFeature("SimulationPoints", "Sim",  "General Distribution", "Location Distribution")
v1 = vec(0, 0, 35); v2 = vec(0, 0, 40); v3 = vec(0, 0, 140); loc.Locations = [v1, v2, v3]

# Mean wind Profile
mean = WindLabObjects.makeFeature("MeanSpeed", "Sim",  "Logarithmic Law Profile", "Mean Wind Profile")
mean.TerrainRoughness = '0.001266 m'; mean.ShearVelocity = '1.76 m/s'

# Frequencies
frequency = WindLabObjects.makeFeature("Frequencies", "Sim",  "Double Index Frequency Discretization", "Frequency Distribution")

# Power Spectrum Density
spectrum = WindLabObjects.makeFeature("Spectrum", "Sim",  "Kaimal Along Wind Spectrum", "Along Wind Spectrum")

# Coherence Function
coherence = WindLabObjects.makeFeature("CoherenceFunction", "Sim",  "Davenport Coherence Function", "Coherence Function")
coherence.ExponentialDecayCz = 10

# Spectrum Decomposition
spectrumD = WindLabObjects.makeFeature("SpectrumDecomposition", "Sim",  "Cholesky Decomposition", "Spectrum Decomposition Method")

# Random phase
randomness = WindLabObjects.makeFeature("RandomPhases", "Sim",  "Uniform Random Phases", "Randomness Provider")

# Simulation method
simMethod = WindLabObjects.makeFeature("SimulationMethod", "Sim",  "Deodatis 1996", "Simulation Method")

# Running the Simulation
velocities = sim.simulate(0)
```

Figure 2: LabRPS Scripting

Plugin System

LabRPS is a highly modular and flexible software platform, structured around a system of workbenches, each designed to cater to specific tasks or workflows in the simulation of random phenomena. These workbenches, which form the core of LabRPS, are composed of plugins which are individual, reusable components that provide specific functionality within each workbench.

In LabRPS, a plugin is a dynamic library that is discovered and loaded at run time as opposed to a dynamic library that an application is linked against at build time. LabRPS Plugins can therefore be written by LabRPS users (Scientists, Engineers, Researchers, Teachers and students) using a well-defined plugin API that is provided. This allows them to extend the functionality of the API in designated ways. The plugins are directly plugged into the corresponding workbench. Plugins are where the most important computations happen. In most of cases, these computations are time-consuming and according to the principles of LabRPS, C++ is recommended for the implementation of such computation tasks. However, a mix of C++ and Python can also be used.

By organizing the software into workbenches and plugins, LabRPS enables efficient, task specific customization, providing users with the flexibility to expand the software's capabilities as required. Plugins can be easily integrated into existing workbenches, allowing for quick implementation of new methods or algorithms without disrupting the overall structure of the software. This architecture ensures that LabRPS remains a powerful yet accessible tool for diverse users with varying needs, from academic researchers developing new simulation techniques to industry professionals seeking realistic numerical models of random phenomena. Figure 3 illustrates the schematic architecture LabRPS structure.

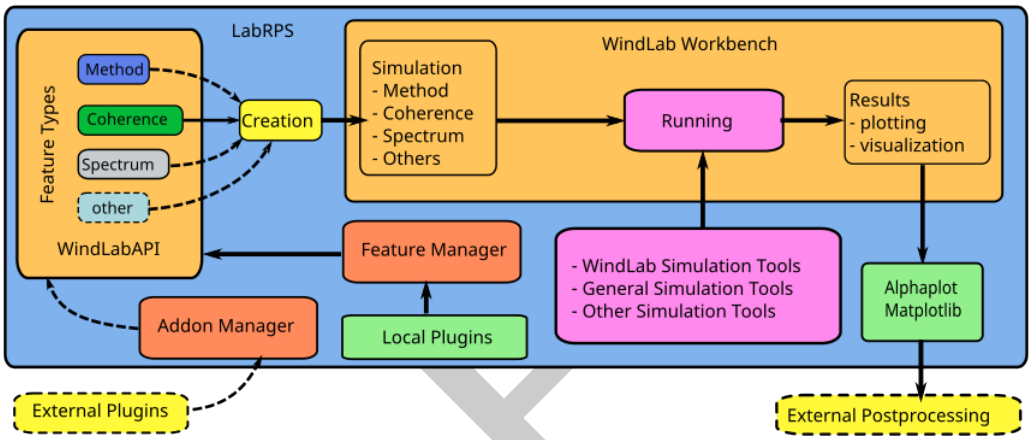

Figure 3: Plugin System

Resources

- Official Website: https://labrps.com
- Documentation: https://wiki.labrps.com
- Download Page: https://sourceforge.net/projects/labrps
- Plugin API Documentation: https://api.labrps.com
- Community Forum: https://labrps.com/boards
- Tutorials: https://wiki.labrps.com/Tutorials
- Plugins: https://wiki.labrps.com/Plugins
- Add your Research: https://wiki.labrps.com/Plugin_Creation

Acknowledgements

We would like to express our sincere gratitude to Siyu Zhu from the College of Environment and Civil Engineering at Chengdu University of Technology, China, for his insightful suggestions and thorough review. We also wish to thank Williams Adilehou and Hermann Sakponou for their continuous support throughout the development of this project.

References

Chen, L., & Letchford, C. W. (2004). A deterministic–stochastic hybrid model of downbursts and its impact on a cantilevered structure. *Engineering Structures*, *26*(5), 619–629. https://doi.org/10.1016/j.engstruct.2003.12.009

Deodatis, G., & Shinozuka, M. (1988). Auto-regressive model for nonstationary stochastic processes. *Journal of Engineering Mechanics*, *114*(11), 1995–2012. https://doi.org/10.1061/(ASCE)0733-9399(1988)114:11(1995)

DRAFT

Grigoriu, M. (1993). On the spectral representation method in simulation. *Probabilistic Engineering Mechanics*, *8*(2), 75–90. https://doi.org/10.1016/0266-8920(93)90002-D

Kareem, A. (2008). Numerical simulation of wind effects: A probabilistic perspective. *Journal of Wind Engineering and Industrial Aerodynamics*, *96*(10), 1472–1497. https://doi.org/10.1016/j.jweia.2008.02.048

Li, Yousun, & Kareem, A. (1990). ARMA systems in wind engineering. *Probabilistic Engineering Mechanics*, *5*(2), 49–59. https://doi.org/10.1016/S0266-8920(08)80001-X

Li, Yousun, & Kareem, A. (1991). Simulation of multivariate nonstationary random processes by FFT. *Journal of Engineering Mechanics*, *117*(5), 1037–1058. https://doi.org/10.1061/(ASCE)0733-9399(1991)117:5(1037)

Li, Yongle, Togbenou, K., Xiang, H., & Chen, N. (2017). Simulation of non-stationary wind velocity field on bridges based on taylor series. *Journal of Wind Engineering and Industrial Aerodynamics*, *169*, 117–127. https://doi.org/10.1016/j.jweia.2017.07.005

MIGNOLET, M., & SPANOS, P. (1989). ARMA monte carlo simulation in probabilistic structural analysis. In *28th structures, structural dynamics and materials conference*. https://doi.org/10.2514/6.1987-932

Shinozuka, M., & Jan, C.-M. (1972). Digital simulation of random processes and its applications. *Journal of Sound and Vibration*, *25*(1), 111–128. https://doi.org/10.1016/0022-460X(72)90600-1

Spanos, P. D., & Zeldin, B. A. (1998). Monte carlo treatment of random fields: A broad perspective. *Applied Mechanics Reviews*, *51*(3), 219–237. https://doi.org/10.1115/1.3098999

Togbenou, K., Li, Y., Chen, N., & Liao, H. (2016). An efficient simulation method for vertically distributed stochastic wind velocity field based on approximate piecewise wind spectrum. *Journal of Wind Engineering and Industrial Aerodynamics*, *151*, 48–59. https://doi.org/10.1016/j.jweia.2016.01.005

Togbenou, K., Xiang, H., Li, Y., & Chen, N. (2018). Improved spectral representation method for the simulation of stochastic wind velocity field based on FFT algorithm and polynomial decomposition. *Journal of Engineering Mechanics*, *144*(2), 04017171. https://doi.org/10.1061/(ASCE)EM.1943-7889.0001400

Yorik, R., & others. (2022). *FreeCAD: An open-source parametric 3D CAD modeler*. https://github.com/FreeCAD/FreeCAD.

DRAFT